Linux Fundamentals — Interview Questions

Q1 — Virtual Filesystems

A junior engineer says `cat /proc/meminfo` reads data from disk just like any other file. Is he correct? Explain what `/proc` actually is, how it works internally, and what the difference between `/proc` and `/sys` is.

> 📄 `01_Linux_Fundamentals.md` → 1.1 Linux Filesystem Hierarchy (FHS) → The virtual filesystems: /proc and /sys

---

Q2 — Disk Space Debugging

`df -h` shows your root filesystem is 100% full, but `du -sh /*` accounts for only 60% of the space. Name the two most likely root causes, explain why each causes this discrepancy, and walk through exactly how you'd diagnose and fix each one.

> 📄 `06_Storage_IO.md` → 6.1 df, du — Disk Usage, Inode Exhaustion → Gotchas

---

Q3 — Privilege Without Root

You need to allow a non-root service to bind to port 80. Give at least two production-viable approaches, explain the trade-offs of each, and state which one you'd recommend for a modern Linux system and why.

> 📄 `01_Linux_Fundamentals.md` → 1.13 Namespaces & Cgroups + `08_Security_Permissions.md` → 8.5 Linux Capabilities

---

Q4 — Unkillable Process

A production process is completely unresponsive. You try `kill -9 <PID>` and nothing happens — the process is still there. What process state is it likely in? Why does SIGKILL not work in this state? What are your diagnostic steps and options?

> 📄 `02_Process_Management.md` → 2.5 Process States — R, S, D, Z, T → State D — Uninterruptible Sleep

---

Q5 — Group Membership Trap

You run `sudo usermod -G docker alice` to add Alice to the docker group. The next day Alice reports she can no longer run sudo commands. What exactly happened at the kernel/system level, how do you verify it, fix it, and what's the correct command you should have used?

> 📄 `01_Linux_Fundamentals.md` → 1.4 Users, Groups & sudo → Gotchas

Q6 — Hard Links vs Soft Links

A colleague asks you to create a backup of `/etc/nginx/nginx.conf` using a hard link. You refuse and suggest a soft link instead. Explain the fundamental difference between hard links and soft links at the inode level, why a hard link would be problematic here, and give two real production scenarios where you'd use each.

> 📄 `01_Linux_Fundamentals.md` → 1.6 Hard Links vs Soft Links — ln, Inodes, How They Differ

---

Q7 — File Descriptor Internals

A server's disk is showing 100% full in `df -h` but you cannot find any large files. A teammate suggests restarting a running application will free up space. Why would restarting an application free disk space, and how would you confirm this is the issue without restarting anything? Can you free the space without a restart?

> 📄 `01_Linux_Fundamentals.md` → 1.8 File Descriptor Internals — stdin/stdout/stderr, /proc/PID/fd → Gotchas

---

Q8 — systemd Service Management

You edit `/lib/systemd/system/nginx.service` directly to increase `LimitNOFILE` to 65536, then run `systemctl restart nginx`. The change has no effect. Walk through exactly why this happened, what the correct approach is, and what command is always mandatory after any unit file change.

> 📄 `01_Linux_Fundamentals.md` → 1.10 systemd & Service Management → Gotchas

Q9 — Boot Process

A remote server is completely inaccessible after a reboot. You have console access via IPMI/KVM. You suspect the root password is unknown and the filesystem may be read-only. Walk through the exact steps to recover access using the GRUB bootloader, including what kernel parameter to add and what commands to run once in the emergency shell.

> 📄 `01_Linux_Fundamentals.md` → 1.11 Linux Boot Process — BIOS/UEFI → GRUB → Kernel → initrd → systemd → Password recovery procedure

---

Q10 — Containers & Kernel Primitives

Your manager asks: *"What actually makes a Docker container different from a regular Linux process?"* Give a precise technical answer covering the two core kernel mechanisms involved, what each one does, and how Docker uses them when you run `docker run`.

> 📄 `01_Linux_Fundamentals.md` → 1.13 Namespaces & Cgroups — Foundation of Containers

---

Q11 — File Permissions Deep Dive

A web application is running as `www-data`. You have a shared directory `/var/project` owned by `alice:engineering`. You need `www-data` to read files, `alice` to read/write, and no access for anyone else — without adding `www-data` to the engineering group. What permission model would you use, walk through the exact commands, and explain the one critical step most engineers forget that causes newly created files to break access.

> 📄 `01_Linux_Fundamentals.md` → 1.3 File Permissions & Ownership + `08_Security_Permissions.md` → 8.3 ACLs — getfacl, setfacl

---

Q12 — SUID/SGID/Sticky Bit

Explain the practical purpose of SUID, SGID, and the sticky bit with one real example of each. Then explain why setting SUID on a Python or Bash script has absolutely no effect on Linux, even though the bit is set and visible in `ls -la`.

> 📄 `01_Linux_Fundamentals.md` → 1.3 File Permissions & Ownership → Special permission bits + Gotchas

---

Q13 — umask in Production

A systemd service is creating log files with permissions `0644` but your security policy requires `0640` so that only the owning group can read them, and others have no access at all. The developer says *"just chmod the files after creation."* Why is that approach fragile? What is the correct solution, and where exactly do you configure it for a systemd service?

> 📄 `01_Linux_Fundamentals.md` → 1.3 File Permissions & Ownership → umask — Default permission mask

---

Q14 — Kernel Modules

You are setting up a new Kubernetes node and the CNI plugin is failing. You suspect `br_netfilter` and `overlay` modules are not loaded. Walk through how you'd check, how you'd load them immediately, and how you'd make them persist across reboots. Also explain the difference between `modprobe` and `insmod`.

> 📄 `01_Linux_Fundamentals.md` → 1.12 Kernel Modules — lsmod, modprobe, insmod, rmmod

---

Q15 — udev & Device Management

A new USB serial device is being assigned a random device name like `/dev/ttyUSB0` or `/dev/ttyUSB1` on every reboot, breaking your automation scripts. Explain how udev works internally, how you would investigate the device's attributes, and how you would write a udev rule to give it a consistent, persistent name like `/dev/mydevice`.

> 📄 `01_Linux_Fundamentals.md` → 1.14 udev & Device Management — How Devices Appear in /dev

---

Q16 — Process Creation Internals

A senior engineer says *"every process on Linux is created the same way — there are no exceptions."* Explain the `fork()` + `exec()` model, what exactly happens at each step, and use a concrete example of what happens under the hood when you type `ls` in a bash shell. Also explain what VSZ and RSS are and which one actually matters for memory monitoring.

> 📄 `02_Process_Management.md` → 2.1 Process Basics — PID, PPID, ps, pstree, Foreground/Background → How processes are created internally + Gotchas

---

Q17 — Zombie & Orphan Processes

During a production incident you notice the process table has thousands of `Z` state processes accumulating over time. Explain exactly what a zombie process is, why it cannot be killed with `kill -9`, what resource it actually consumes, what the real danger is if they keep accumulating, and walk through your exact steps to clean them up without rebooting.

> 📄 `02_Process_Management.md` → 2.9 Zombie & Orphan Processes — How They Form, How to Clean Them

---

Q18 — Signals Deep Dive

You need to gracefully shut down a production database process. Walk through the correct signal sequence you'd use, why you should never reach for SIGKILL first, what happens internally when each signal is sent, and explain why SIGKILL cannot be caught, blocked, or ignored while SIGTERM can. Also — how do you check if a process is alive without sending it a real signal?

> 📄 `02_Process_Management.md` → 2.4 Signals & kill — SIGTERM, SIGKILL, SIGHUP, SIGINT, Trapping Signals

---

Q19 — Load Average Misconception

A monitoring alert fires: *"Load average is 24 on this server."* A junior engineer immediately concludes the CPUs are maxed out. Is he necessarily correct? Explain precisely what load average measures, why high load with low CPU utilization is possible, how you'd distinguish CPU pressure from I/O pressure, and what tool you'd use to confirm your diagnosis.

> 📄 `07_Performance_Observability.md` → 7.4 Load Average — What 1/5/15 Minute Values Mean + `02_Process_Management.md` → 2.3 top and htop

---

Q20 — strace for Production Debugging

A microservice is silently failing — no useful logs, no obvious errors. You need to understand what it's actually doing at runtime. Walk through how you'd use `strace` to diagnose it, what specific flags you'd use for each scenario: (a) it's hanging, (b) it can't find a config file, (c) a network connection is failing. Also — what is the performance overhead concern and how do you mitigate it?

> 📄 `02_Process_Management.md` → 2.10 strace & ltrace — Syscall Tracing for Live Debugging

---

Q21 — /proc/PID Internals

A production Java service is suspected of having a file descriptor leak — over time it becomes unresponsive and eventually crashes. Walk through exactly how you'd confirm the leak using `/proc/PID` internals, which specific files you'd check, how you'd compare against the process's limits, and how you'd identify what type of file descriptor is leaking. Also — what's the classic *"disk full but no files found"* scenario and how does `/proc/PID/fd` help solve it?

> 📄 `02_Process_Management.md` → 2.7 /proc/PID Internals — cmdline, maps, fd, status, limits

---

Q22 — CPU Scheduler & cgroups

A containerized application is running slowly despite the host having plenty of available CPU. Your colleague checks `top` inside the container and sees CPU usage is only 40%. He concludes the app is not CPU-bound. You disagree. Explain the concept of CPU throttling via cgroups, how `cpu.cfs_quota_us` and `cpu.cfs_period_us` work, why the container appears to have low CPU usage while actually being throttled, and exactly where you'd look to confirm throttling is happening.

> 📄 `02_Process_Management.md` → 2.8 Linux Scheduler — CFS, Time Slices, cgroups CPU Limits → cgroups — CPU limiting for containers + Gotchas

---

Q23 — lsof in Production

You receive an alert: *"Port 8080 is already in use, service failed to start."* Walk through using `lsof` to diagnose this, explain the key flags you'd use and why. Then separately — explain how you'd use `lsof` to find all `CLOSE_WAIT` socket connections and what `CLOSE_WAIT` accumulation tells you about the application.

> 📄 `02_Process_Management.md` → 2.11 lsof — Open Files, Socket States, Debugging FD Leaks

---

Q24 — grep in Production Log Analysis

You're on-call at 2 AM. Users are reporting 500 errors. You have access to `/var/log/nginx/access.log`. Walk through the exact `grep` commands you'd run to: (a) count how many 500 errors occurred, (b) find which endpoints are failing, (c) identify which client IPs are hitting the 500s, and (d) see what happened in the logs just before each 500 error. Explain the specific flags used in each case.

> 📄 `04_Text_Processing.md` → 4.1 grep — Search Tool → Real-world production example + Key flags

---

Q25 — awk vs sed vs cut

You have an nginx access log and need to: (a) extract only the IP addresses, (b) find the top 5 most frequent IPs, (c) replace all occurrences of `staging` with `production` in a config file in-place. For each task, state which tool you'd use, write the exact command, and explain why you chose that tool over the alternatives.

> 📄 `04_Text_Processing.md` → 4.6 grep vs sed vs awk — When to Use Which + 4.2 cut, sort, uniq, wc + 4.4 sed — Stream Editor

---

Q26 — awk Deep Dive

You have an nginx access log with the standard format. Write a single `awk` program that: (a) counts total requests per HTTP status code, (b) calculates the average response size, (c) prints a formatted summary report with headers at the end. Explain the role of `BEGIN`, `END`, associative arrays, and `NR` vs `NF` in your solution.

> 📄 `04_Text_Processing.md` → 4.5 awk — Field Processor & Mini Language + 4.7 Advanced awk — Multi-rule, Associative Arrays, Formatted Output

---

Q27 — sed Advanced

A deployment script needs to: (a) update `DB_HOST=localhost` to `DB_HOST=prod-db.internal` in `/etc/app/config.env` in-place with a backup, (b) remove all comment lines and blank lines from an nginx config, (c) extract only the lines between `BEGIN CERTIFICATE` and `END CERTIFICATE` markers in a PEM file. Write the exact `sed` commands and explain one critical portability gotcha between GNU/Linux and macOS sed.

> 📄 `04_Text_Processing.md` → 4.4 sed — Stream Editor → In-place edit, deletion, print with -n, Gotchas

---

Q28 — jq for Cloud Operations

You're writing a deployment script that calls `aws ec2 describe-instances`. The JSON response contains nested arrays. Walk through how you'd: (a) extract only running instance IDs and their private IPs, (b) filter instances where CPU > 80, (c) use the output in a bash loop to SSH into each instance. Explain why the `-r` flag is critical when using `jq` output in shell scripts.

> 📄 `04_Text_Processing.md` → 4.9 jq — JSON Parsing for Cloud APIs

Q29 — DNS Resolution Chain

A microservice inside a Kubernetes pod is making API calls to `api.github.com` and experiencing 3x higher latency than expected on DNS resolution. You suspect the Kubernetes DNS configuration is the culprit. Explain the full DNS resolution chain inside a pod, what `ndots:5` means, exactly why external FQDNs like `api.github.com` cause extra DNS lookups, how many wasted queries happen, and what the fix is.

> 📄 `05_Networking.md` → 5.7 DNS Resolution Chain — /etc/resolv.conf, /etc/hosts, nsswitch.conf → DNS in Kubernetes — full detail

---

Q30 — TCP States in Production

Your monitoring shows a microservice has 847 `CLOSE_WAIT` connections that keep growing over time, while `TIME_WAIT` connections number in the thousands on a different service. Your manager asks you to fix both with sysctl tuning. Explain: why `CLOSE_WAIT` cannot be fixed with sysctl, what it actually indicates, what the real fix is — and separately, whether thousands of `TIME_WAIT` connections are actually a problem and when they become one.

> 📄 `05_Networking.md` → 5.9 TCP States — SYN_SENT, TIME_WAIT, CLOSE_WAIT → CLOSE_WAIT — the dangerous one + TIME_WAIT + Gotchas

---

Q31 — SSH Internals & Security

A security audit flags two issues on your servers: (1) SSH agent forwarding (`-A`) is being used to access internal servers via a bastion host, and (2) sshd allows password authentication. Explain exactly why agent forwarding is a security risk compared to `ProxyJump`, how `ProxyJump` works differently at the protocol level, and walk through the exact `~/.ssh/config` changes and `sshd_config` hardening steps you'd implement. Also — what is the one sshd operation that must never be `restart` and why?

> 📄 `05_Networking.md` → 5.8 ssh Internals — Key Exchange, Tunnels, ~/.ssh/config, ProxyJump → SSH security hardening + Gotchas

---

Q32 — iptables & NAT

You need to set up a Linux server as a NAT gateway so that instances on a private subnet `10.0.0.0/8` can reach the internet through `eth0`. Walk through the exact `iptables` commands needed, explain which table and chain each rule goes into and why. Then separately — explain the single most common iptables mistake that causes all inbound responses to be silently dropped after setting a default DROP policy.

> 📄 `05_Networking.md` → 5.5 iptables / nftables — Firewall Rules, Chains, NAT → NAT rules + Gotchas

---

Q33 — tcpdump for Incident Response

During a production incident you need to capture network traffic without impacting the server. Walk through your exact `tcpdump` command including all flags you'd use to: minimize performance impact, save to a file for offline analysis, exclude SSH noise, and capture only traffic to/from port 443. Then explain how you'd analyze the capture to find TCP connection resets and what RST packets indicate in production.

> 📄 `05_Networking.md` → 5.4 tcpdump — Packet Capture → Real production examples + Gotchas

---

Q34 — LVM Online Operations

A production database server is running out of disk space on `/var/lib/postgresql` which is mounted on an LVM logical volume. You have a new `/dev/sdc` disk available. Walk through the exact LVM commands to add the new disk and expand the filesystem without any downtime. Then explain the one filesystem type that cannot be shrunk on LVM and why that matters for planning.

> 📄 `06_Storage_IO.md` → 6.5 LVM — Physical Volumes, Volume Groups, Logical Volumes → Extending (online!) + Gotchas

---

Q35 — I/O Performance Diagnosis

A server's load average spikes to 40 every night at 2 AM but CPU utilization stays below 10%. Walk through your complete diagnostic process: which tools you'd use in which order, what specific metrics you'd look for in each tool's output, how you'd identify which process is causing the I/O, and how you'd determine which files it's accessing. Explain why high load with low CPU is possible and what it tells you.

> 📄 `06_Storage_IO.md` → 6.6 iostat, iotop — I/O Performance Analysis + `07_Performance_Observability.md` → 7.4 Load Average + 7.1 vmstat

---

Q36 — Filesystem Selection Trade-offs

Your team is provisioning storage for three different workloads: (a) a PostgreSQL database with heavy random small writes, (b) a video streaming server handling large sequential reads, (c) a NAS backup server where point-in-time snapshots are critical. For each workload, recommend a filesystem, explain your reasoning, and explain one specific gotcha of your chosen filesystem that could hurt you in production if ignored.

> 📄 `06_Storage_IO.md` → 6.3 Filesystem Types — ext4, xfs, btrfs, tmpfs — Trade-offs

---

Q37 — RAID Trade-offs

Your manager asks you to choose a RAID level for a new database server with 6 identical 4TB disks. He wants maximum fault tolerance but is concerned about write performance. Walk through RAID 5, RAID 6, and RAID 10 — compare usable capacity, fault tolerance, write performance, and rebuild risk for each. Which would you recommend and why? Also explain the specific danger that makes RAID 5 particularly risky with large modern HDDs.

> 📄 `06_Storage_IO.md` → 6.7 RAID Concepts — RAID 0/1/5/6/10 → RAID comparison table + Gotchas

---

Q38 — Page Cache Misconception

A junior engineer monitors a production server and panics: *"We only have 200MB of free memory left on a 64GB server — we're about to OOM!"* You tell him he's misreading the output. Explain exactly what `free -h` is showing, why low free memory is completely normal and expected on Linux, what the correct metric to watch is, and what the actual danger signs of genuine memory pressure look like. Include what `vmstat` fields confirm active memory pressure.

> 📄 `06_Storage_IO.md` → 6.9 Page Cache & Buffer Cache — How Linux Uses RAM for I/O + `07_Performance_Observability.md` → 7.3 free — Memory, Buffers, Cache — What's Actually "Free"

Q39 — USE Method in Practice

You are the on-call SRE and receive an alert: *"API response times have increased 4x in the last 10 minutes."* Walk through the USE Method framework systematically — what it stands for, how you'd apply it to CPU, memory, disk, and network in order, which specific commands you'd run for each resource's utilization, saturation, and errors, and how this approach is superior to randomly checking metrics.

> 📄 `07_Performance_Observability.md` → 7.6 USE Method — Utilization, Saturation, Errors Framework

---

Q40 — fio Benchmarking

Before migrating a MySQL database to a new server, you want to validate the storage can handle the expected workload. Explain what three key benchmarks you'd run with `fio`, the exact flags for each, why `--direct=1` is non-negotiable, and how you'd interpret the results — specifically what IOPS, throughput, and P99 latency values you'd expect from SATA SSD vs NVMe, and what metric matters most for database transaction performance.

> 📄 `06_Storage_IO.md` → 6.10 fio — I/O Benchmarking in Production

Q41 — perf & Flamegraphs

A Python web service is consuming 95% CPU on one core but the application logs show nothing unusual. Your colleague suggests adding print statements to find the bottleneck. Explain why that approach is flawed and walk through exactly how you'd use `perf` to diagnose this — which commands, which flags, and how you'd generate a flamegraph. Explain how to read a flamegraph — what the X axis, Y axis, and width of frames represent — and what a *"single-threaded bottleneck"* looks like in `mpstat` output vs `vmstat`.

> 📄 `07_Performance_Observability.md` → 7.5 perf Basics — CPU Profiling, Flamegraphs + 7.2 mpstat, iostat, sar — The sysstat Suite

---

Q42 — sar for Historical Analysis

A production incident happened yesterday at 2 PM but by the time you're investigating it's 9 AM the next day — all real-time tools show nothing. Walk through exactly how you'd use `sar` to reconstruct what happened: which commands, which flags, which files, what each metric tells you. Explain what needs to be configured for sar historical data to even exist, and what you'd check first to determine if the incident was CPU, memory, I/O, or network related.

> 📄 `07_Performance_Observability.md` → 7.2 mpstat, iostat, sar — The sysstat Suite → sar — The historical recorder

---

Q43 — OOM Killer Internals

At 3 AM your monitoring alerts that a critical service was killed. `dmesg` shows an OOM kill event. Walk through: how the OOM killer decides which process to kill, how you'd read the `dmesg` OOM log entry to understand what happened, how you'd protect your critical service from being killed in the future, and what the difference is between `oom_score` and `oom_score_adj`. Also — what is the Kubernetes equivalent of OOM protection?

> 📄 `07_Performance_Observability.md` → 7.8 OOM Killer — How Linux Kills Processes Under Memory Pressure

---

Q44 — eBPF & Production Tracing

A microservice is making unexpected outbound connections to an unknown IP at random intervals. Traditional tools like `netstat` and `ss` miss it because the connections are very short-lived. Walk through exactly which eBPF-based tool you'd use, the exact command, what output it produces, and why it catches short-lived connections that `ps` and `ss` miss. Then explain at a high level how eBPF works internally and why it's safe to run in production unlike `strace`.

> 📄 `07_Performance_Observability.md` → 7.9 systemtap & eBPF/bpftrace — Production Tracing + 7.10 Brendan Gregg's Tools + `05_Networking.md` → 5.11 eBPF Basics

---

Q45 — SELinux Troubleshooting

You deploy a new nginx configuration that serves files from `/data/www/` instead of the default `/var/www/html/`. Everything looks correct — permissions are fine, nginx config is valid — but nginx returns 403 for all requests. You suspect SELinux. Walk through your complete diagnostic and fix process: how you confirm SELinux is the cause, how you identify the exact denial, the correct fix using `semanage` and `restorecon`, and why `setenforce 0` is not an acceptable production fix.

> 📄 `08_Security_Permissions.md` → 8.4 SELinux / AppArmor — MAC, Contexts, Modes, Troubleshooting → Fixing "permission denied" with SELinux

---

Q46 — PAM Authentication Chain

Your security team mandates: (a) accounts must be locked after 5 failed login attempts for 15 minutes, (b) passwords must be minimum 12 characters with at least 3 character classes, (c) resource limits must be enforced at login. Walk through exactly which PAM modules handle each requirement, where they go in the PAM stack, what control flags you'd use and why, and explain one critical gotcha about PAM misconfiguration that could lock everyone out of the server.

> 📄 `08_Security_Permissions.md` → 8.2 PAM — Pluggable Authentication Modules → Important PAM modules + Gotchas

---

Q47 — Linux Capabilities vs Setuid

Your team has a custom Go binary that needs to bind to port 443 and capture raw packets for a network monitoring tool. A colleague suggests making it setuid root. Explain exactly why that's dangerous, what the correct approach is using Linux capabilities, the exact commands to apply them, and how you'd verify they're set correctly. Also explain why `CAP_SYS_ADMIN` should be treated almost identically to full root access.

> 📄 `08_Security_Permissions.md` → 8.5 Linux Capabilities — Replacing Setuid Root

Q48 — auditd for Compliance

A compliance audit requires: (a) log every time `/etc/sudoers` is modified, (b) log every command executed as root by a non-root user, (c) ensure logs cannot be tampered with. Walk through the exact `auditctl` rules for each requirement, explain the `auid` field and why it's critical for tracing actions back to the original user even after `sudo su -`, and explain how you'd make audit rules immutable and why remote logging matters.

> 📄 `08_Security_Permissions.md` → 8.7 Audit Framework — auditd, Tracking Privileged Access

---

Q49 — cron Production Pitfalls

A cron job that runs perfectly when executed manually fails silently every night. Walk through the five most likely causes in order of probability, the exact diagnostic steps for each, and explain why `crontab -r` is one of the most dangerous commands a sysadmin can accidentally run. Also — you have 500 servers all running the same cron job at exactly 2:00:00 AM hitting a shared database. What's the problem and what are your solutions using both cron and systemd timers?

> 📄 `09_Scheduling_Automation.md` → 9.1 cron & crontab — Syntax, Fields, Common Pitfalls → Gotchas + 9.3 systemd Timers → RandomizedDelaySec

---

Q50 — systemd Timers vs cron

Your team is migrating all cron jobs to systemd timers. A colleague asks: *"Why bother? Cron works fine."* Make the technical case for systemd timers covering at least 4 concrete advantages with specific examples. Then walk through creating a complete systemd timer that runs a database backup daily at 2:30 AM, catches missed runs if the server was off, spreads execution across a 15-minute window on a fleet, limits CPU to 25% and memory to 512MB, and explain every mandatory step including what happens if you forget `daemon-reload`.

> 📄 `09_Scheduling_Automation.md` → 9.3 systemd Timers — Modern Replacement for Cron → systemd timers vs cron + Creating a systemd timer + Gotchas

---

Q51 — File & Directory Operations

A junior engineer runs `rm -rf / tmp/*` intending to clean `/tmp` but accidentally destroys the root filesystem. Explain exactly what went wrong at the shell parsing level. Then walk through the safe alternatives — specifically how you'd use `find` with `-delete` instead of `rm -rf` to safely remove log files

older than 30 days from `/var/log`, why you should always preview before deleting, and explain the difference between `find -exec cmd {} \;` and `find -exec cmd {} +` in terms of performance.

> 📄 `01_Linux_Fundamentals.md` → 1.2 File & Directory Operations — ls, cp, mv, rm, find, locate → rm + find + Gotchas

---

Q52 — Text Viewing & tail Behaviour

A production nginx log is being actively rotated by `logrotate` every hour. You have two engineers monitoring it — one using `tail -f` and one using `tail -F`. After log rotation, one of them stops seeing new log entries. Which one stops and exactly why? Explain the internal difference between `-f` and `-F`, when vim is unavoidable on a server and the absolute minimum commands you must know, and why `cat` on a large log file is dangerous.

> 📄 `01_Linux_Fundamentals.md` → 1.5 Text Viewing & Editing — cat, less, head, tail, vim, nano → tail -f vs tail -F + Gotchas

---

Q53 — Job Control & nohup vs disown

You SSH into a production server and start a long-running data migration script in the foreground. Halfway through you realise your SSH session will time out before it finishes. You cannot restart the script. Walk through exactly what happens to your script when the SSH session closes, the difference between `nohup` and `disown` at the signal level, how you'd rescue the already-running process without restarting it, and when you'd use each approach going forward.

> 📄 `02_Process_Management.md` → 2.2 Job Control — fg, bg, jobs, &, nohup, disown → nohup + disown + Gotchas

---

Q54 — top/htop Deep Read

You open `top` on a busy production server and see: load average `14.23, 12.87, 9.65` on a 4-core machine, `%Cpu: 2.3 us, 1.1 sy, 84.3 wa`, and one process showing `%CPU 400%`. Walk through what each of these three observations tells you independently, what the `wa` value definitively tells you about where the bottleneck is, why 400% CPU on a single process is not a bug, and what keyboard shortcut reveals whether the bottleneck is on one specific core or spread across all cores.

> 📄 `02_Process_Management.md` → 2.3 top and htop — Reading Load Average, CPU, Memory Columns

---

Q55 — nice & renice in Production

Your team runs nightly batch data processing jobs on the same servers that serve live traffic. During business hours the batch job is starving the web application of CPU. Walk through exactly how `nice` and `renice` work internally via the CFS scheduler, the valid range of values and what they mean, the exact commands to deprioritize the batch job and boost the web server, and the one permission rule that prevents non-root users from raising their own process priority.

> 📄 `02_Process_Management.md` → 2.6 nice & renice — CPU Scheduling Priority → Real-world patterns + Gotchas

---

Q56 — cut, sort, uniq, wc Pipeline

You have an nginx access log and need to answer three questions using only pipeline tools: (a) how many unique IP addresses hit your server today, (b) what are the top 5 most frequent HTTP status codes and their counts, (c) which hour of the day had the most requests. Write the exact pipeline for each, explain why `uniq` alone without `sort` will give wrong results, and explain the critical difference between `sort -n` and `sort` for numeric data with a concrete example showing where alphabetic sort breaks.

> 📄 `04_Text_Processing.md` → 4.2 cut, sort, uniq, wc — Pipeline Building Blocks → The Power Pipeline Pattern + Gotchas

---

Q57 — tr & Log Cleaning

A CSV file exported from Windows has `\r\n` line endings breaking your Linux pipeline tools. A config file has inconsistent multiple spaces between values. A script needs to extract only numeric characters from mixed output like `"Server uptime: 3652 days"`. Write the exact `tr` command for each scenario. Then explain the single most important gotcha about `tr` that catches engineers who confuse it with `sed` — specifically what `tr 'hello' 'world'` actually does versus what most people think it does.

> 📄 `04_Text_Processing.md` → 4.3 tr — Character Translation → Real production use + Gotcha

---

Q58 — Log Parsing Pipelines for Incident Analysis

It's 2 AM and you suspect a brute force attack on SSH. You have access to `/var/log/auth.log`. Write the exact pipeline to: (a) find the top 10 IPs with the most failed password attempts, (b) identify which usernames are being targeted most, (c) check if any of those IPs subsequently had a successful login. Then explain how you'd use `awk` instead of `grep + cut` for the same analysis and why `awk` is superior for multi-field log analysis.

> 📄 `04_Text_Processing.md` → 4.8 Log Parsing Pipelines — Real Incident Analysis → Scenario 4: Auth log analysis

---

Q59 — ping, traceroute, dig Toolkit

A service is unreachable from your application server. Walk through your exact diagnostic sequence using `ping`, `traceroute`, and `dig` in order — what each tool tests, what a specific failure in each tells you, and what false negatives to watch for. Specifically: why can ping fail while HTTP works fine, what does `* * *` in traceroute actually mean and is it always a problem, and what is the difference between `dig @8.8.8.8 domain` vs `dig domain` and when does that difference matter critically in production?

> 📄 `05_Networking.md` → 5.1 ping, traceroute, dig, nslookup — Basic Connectivity → Gotchas + Reading traceroute

---

Q60 — ss & netstat for Socket Diagnosis

A service fails to start with *"address already in use"* on port 5432. Walk through the exact `ss` commands to diagnose this. Then explain: what `Recv-Q` being non-zero on a `LISTEN` socket means versus on an `ESTABLISHED` socket — these are two completely different problems. Finally explain why `ss` is preferred over `netstat`, what the key architectural difference is, and what `ss -tulnp` expands to and why each flag matters.

> 📄 `05_Networking.md` → 5.2 netstat / ss — Socket States, Listening Ports → Production use cases + Gotchas

---

Q61 — curl & wget in Production Scripts

A deployment script uses `curl https://api.example.com/health` to check if a service is ready before proceeding. A colleague says *"it works fine — curl exits 0 when the service responds."* You identify a critical bug. Explain exactly what the bug is, which specific flag fixes it and why, and walk through writing a robust health check function using `curl` that: handles timeouts, captures HTTP status code separately from body, retries with backoff, and works correctly with `set -e` in a bash script.

> 📄 `05_Networking.md` → 5.3 curl & wget — HTTP from the Command Line → Gotchas + Production patterns

---

Q62 — ip addr, ip route, ip link

A newly provisioned server cannot reach `8.8.8.8` but can ping its own IP. Walk through the exact diagnostic commands using only the `ip` toolset — no `ifconfig`, no `route` — to check: interface state, IP assignment, routing table, and ARP table. Explain the difference between `UP` and `LOWER_UP` flags, what `ip route get 8.8.8.8` tells you that `ip route show` alone does not, and why all `ip` commands are non-persistent and what you must do to make changes survive reboot.

> 📄 `05_Networking.md` → 5.6 Network Interfaces — ip addr, ip route, ip link → Gotchas + ip neigh

---

Q63 — Network Namespaces & Container Networking

A developer asks: *"When I run `docker run -p 8080:80 nginx`, how does traffic actually get from my laptop's port 8080 into the container's nginx on port 80?"* Give a precise technical answer tracing the full packet path — covering network namespaces, veth pairs, the docker0 bridge, iptables DNAT rules, and NAT. Then explain why `127.0.0.1` inside a container does NOT refer to the host, and how you'd debug the container's network from the host without `docker exec`.

> 📄 `05_Networking.md` → 5.10 Network Namespaces — Container Isolation → How Docker uses namespaces end-to-end + Gotchas

---

Q64 — mount, fstab & Boot Safety

A junior engineer adds a new NFS mount to `/etc/fstab` and reboots the server. The server fails to boot because the NFS server is temporarily unreachable. Explain exactly why this happens, which two fstab options would have prevented it, and what the `_netdev` option specifically does. Then explain why you should always use UUIDs instead of device names like `/dev/sda1` in fstab, the exact command to find a device's UUID, and the one mandatory test command before rebooting after any fstab change.

> 📄 `06_Storage_IO.md` → 6.2 mount / umount — Mounting Filesystems, /etc/fstab → fstab + Gotchas

---

Q65 — lsblk, blkid, fdisk, parted

You have a brand new 2TB `/dev/sdb` disk that needs to be partitioned, formatted as XFS, mounted persistently at `/data`, and survive reboots with the correct filesystem. Walk through every single step from scratch using the correct modern tools — explain why you'd choose `parted` over `fdisk` for scripting, why you start the first partition at 1MiB instead of sector 0, what `partprobe` does and when it's needed, and write the complete fstab entry using best practices.

> 📄 `06_Storage_IO.md` → 6.4 lsblk, blkid, fdisk, parted — Block Device Management → Full disk provisioning workflow

---

Q66 — I/O Schedulers

Your team is provisioning two servers: one running PostgreSQL on SATA SSDs, another running a log aggregation workload on spinning HDDs. A colleague says *"just leave the I/O scheduler at the kernel default — it doesn't matter."* Explain what an I/O scheduler actually does and why it matters differently for HDDs vs SSDs, which scheduler you'd choose for each workload and why, the exact commands to check and change the scheduler, and how you'd make the change persist across reboots. Also explain why `none` is often the right choice for VMs in cloud environments.

> 📄 `06_Storage_IO.md` → 6.8 I/O Schedulers — cfq, deadline, noop, mq-deadline → Recommended scheduler by device type + Gotchas

---

Q67 — vmstat Deep Read

You run `vmstat 2` on a production server and see these values across multiple lines: `r=0, b=12, swpd=2048000, si=512, so=1024, wa=84, us=3, sy=2`. Walk through what each of these values tells you independently, build a complete diagnosis from this single output, explain why the first line of `vmstat` output should always be ignored, and explain what the `st` column means and when it becomes a serious concern specifically for cloud VMs.

> 📄 `07_Performance_Observability.md` → 7.1 vmstat — Memory, Swap, CPU Overview → Reading the story from vmstat + Gotchas

---

Q68 — dmesg for Hardware Diagnosis

At 3 AM your monitoring shows intermittent disk I/O errors on a production server. Walk through exactly how you'd use `dmesg` to investigate — the specific flags for readable timestamps and filtering by severity, the exact grep patterns for disk errors vs hardware MCE errors vs OOM events, what a repeating `blk_update_request: I/O error` message tells you versus a single occurrence, and what your immediate next steps are after confirming disk errors in dmesg. Also explain why `journalctl -k -b -1` is sometimes more useful than `dmesg`.

> 📄 `07_Performance_Observability.md` → 7.7 dmesg — Kernel Ring Buffer, OOM Killer Logs → Critical patterns to recognize + Core usage

---

Q69 — sudo Internals & visudo

Your security team requires: (a) the `deploy` user can restart only nginx and reload its config — nothing else, (b) the `jenkins` user can run docker commands without a password prompt, (c) no one should ever be able to use `sudo vim` or `sudo less` as a privilege escalation path. Walk through the exact sudoers entries for each requirement, explain why `sudo vim` is a security vulnerability, what `sudoedit` is and why it's safer, the exact syntax difference between `/etc/sudoers` and `/etc/sudoers.d/` files, and what single command validates sudoers syntax without applying changes.

> 📄 `08_Security_Permissions.md` → 8.1 sudo Internals — /etc/sudoers, NOPASSWD, visudo → Command argument control + Gotchas

---

Q70 — Seccomp & at/batch

Two-part question covering the final missing sections:

Part A — Seccomp: Docker's default seccomp profile blocks certain syscalls. Name four of the most critical blocked syscalls, explain exactly why each is dangerous in a container context, and explain the difference between `SCMP_ACT_ERRNO` and `SCMP_ACT_KILL` as default actions. When would you use `--security-opt seccomp=unconfined` and what risk does that carry?

Part B — at & batch: Explain the key difference between `at` and `batch`, give a concrete production scenario where `batch` is the better choice over both `at` and cron, and explain the one critical advantage `at` has over cron in terms of environment that makes it more reliable for one-off tasks.

> 📄 `08_Security_Permissions.md` → 8.6 Seccomp — Syscall Filtering + `09_Scheduling_Automation.md` → 9.2 at & batch — One-Off Job Scheduling

---

Q71 — Disk & Filesystem Management Overview

A new engineer asks: *"We already have `df`, `du`, `lsblk`, `fdisk`, and `mount` — what does section 1.7 add that those don't cover?"* Use this as a prompt to explain the relationship between block devices, partitions, filesystems, and mount points as a complete mental model. Specifically: what is the difference between `fdisk -l` and `lsblk`, when would you use `blkid` vs `lsblk -f`, what does `mkfs.xfs` actually do to a partition, and walk through the minimum viable sequence of commands to take a raw blank disk `/dev/sdb` and make it usable at `/mnt/data` — including making it survive a reboot.

> 📄 `01_Linux_Fundamentals.md` → 1.7 Disk & Filesystem Management

Q72 — perl One-liners

You need to: (a) do an in-place config file substitution that works identically on both Linux and macOS without changing your script — your `sed -i` approach keeps breaking on Mac, (b) strip ANSI color codes from a log file that `sed` struggles with due to escape sequence complexity, (c) extract all email addresses from a large file using a regex feature that `grep -E` and `awk` cannot cleanly handle. Write the exact perl one-liner for each, explain which specific perl flags you're using and what each does, and state precisely when you should reach for perl over awk and sed.

> 📄 `04_Text_Processing.md` → 4.10 perl One-liners — When awk/sed Aren't Enough